EMBEDDED SYSTEMS

INTERVIEW PREPARATION ROADMAP

Complete Study Guide with Questions & Answers

TABLE OF CONTENTS

1. C Programming Fundamentals
2. Pointers and Memory Management
3. Bitwise Operations
4. Data Structures for Embedded Systems
5. Microcontroller Basics
6. RTOS Fundamentals
7. Communication Protocols
8. Device Drivers
9. Debugging Techniques
10. Advanced Topics

1. C PROGRAMMING FUNDAMENTALS

Why C for Embedded Systems?

C is the foundation of embedded programming because it provides low-level hardware access while maintaining portability and efficiency. Unlike higher-level languages, C allows direct memory manipulation, bit-level operations, and minimal runtime overhead - all critical for resource-constrained embedded systems.

Q1.1: Explain the difference between #define and const

Answer:
#define is a preprocessor directive that performs text substitution before compilation. It doesn't allocate memory and cannot be type-checked.

const is a type-safe variable that allocates memory and can be debugged. It's preferred in modern C.

Example:
```
#define MAX_SIZE 100 // No type safety, no memory
const int max_size = 100; // Type-safe, has memory address

// In embedded systems, use const for:
const uint32_t PERIPHERAL_BASE = 0x40000000; // Hardware addresses
const uint8_t CONFIG_VALUE = 0xA5; // Configuration values
```

Q1.2: What is the difference between static and extern keywords?

Answer:
static: Limits scope to current file/function. Retains value between function calls.
extern: Declares a variable defined in another file.

Example:
```
// file1.c
static int counter = 0; // Only visible in file1.c
int global_var = 10; // Can be accessed from other files

// file2.c
extern int global_var; // Access global_var from file1.c
// extern int counter; // ERROR: counter is static
```

Q1.3: Explain storage classes in C

Answer:

Storage Class	Lifetime	Scope	Default Value
auto	Block	Block	Garbage
static	Program	File/Function	0
extern	Program	Global	0
register	Block	Block	Garbage

Q1.4: What is structure padding and how does it affect embedded systems?

Answer:
Structure padding is extra bytes added by the compiler to align data members according to the processor's word size. This affects memory usage and hardware register access in embedded systems.

Example:

```
struct Example {
char a; // 1 byte
// 3 bytes padding
int b; // 4 bytes
char c; // 1 byte
// 3 bytes padding
}; // Total: 12 bytes instead of 6

// To prevent padding (for hardware registers):
struct __attribute__((packed)) HardwareReg {
uint8_t control;
uint32_t data;
uint8_t status;
}; // Total: 6 bytes, no padding
```

Q1.5: Explain endianness and its importance in embedded systems

Answer:
Endianness defines the byte order for multi-byte data types:
Big-Endian: Most significant byte stored at lowest address (network protocols)
Little-Endian: Least significant byte stored at lowest address (x86, ARM Cortex-M)

Example:

```
uint32_t value = 0x12345678;

Little-Endian memory: 78 56 34 12
Big-Endian memory: 12 34 56 78

// Check endianness:
int check_endian() {
uint32_t i = 1;
return (*(uint8_t*)&i == 1) ? 0 : 1; // 0=little, 1=big
}
```

2. POINTERS AND MEMORY MANAGEMENT

Critical Concept

Pointers are the heart of embedded systems programming. They enable direct hardware access, efficient data structures, and memory-mapped I/O operations. Mastering pointers is non-negotiable for embedded interviews.

Q2.1: Explain different types of pointers used in embedded systems

Answer:
1. **Normal Pointer**: Points to data in RAM
2. **Pointer to Constant**: Data cannot be modified
3. **Constant Pointer**: Pointer address cannot change
4. **Void Pointer**: Generic pointer, needs casting
5. **Function Pointer**: Points to executable code
6. **NULL Pointer**: Points to nothing (0x00000000)

Example:
```
int value = 10;
int *ptr = &value; // Normal pointer
const int *ptr_const = &value; // Pointer to constant
int *const const_ptr = &value; // Constant pointer
void *void_ptr = &value; // Void pointer

// For hardware registers (constant address):
volatile uint32_t *const GPIO_PORT = (uint32_t*)0x40020000;
```

Q2.2: What is a dangling pointer and how do you prevent it?

Answer:
A dangling pointer points to memory that has been freed or is out of scope. It's a major source of bugs in embedded systems.

Example Problem:
```
int* create_array() {
int arr[10]; // Local array
return arr; // DANGER: Returns pointer to local variable
}

int *ptr = create_array();
ptr[0] = 5; // Undefined behavior - dangling pointer!
```

Solution:
```
// Option 1: Use static
int* create_array() {
static int arr[10]; // Static storage
return arr; // Safe
}

// Option 2: Dynamic allocation
int* create_array() {
int *arr = malloc(10 * sizeof(int));
return arr; // Caller must free()
}

// Option 3: Pass buffer
void fill_array(int *arr, size_t size) {
// Caller manages memory
}
```

Q2.3: How do you access hardware registers using pointers?

Answer:
Hardware registers are accessed through memory-mapped I/O using volatile pointers.

Example - STM32 GPIO:

```
// Define register addresses
#define GPIOA_BASE 0x40020000
#define GPIO_MODER 0x00
#define GPIO_ODR 0x14

// Create volatile pointers
volatile uint32_t *GPIOA_MODER = (uint32_t*)(GPIOA_BASE + GPIO_MODER);
volatile uint32_t *GPIOA_ODR = (uint32_t*)(GPIOA_BASE + GPIO_ODR);

// Configure pin 5 as output
*GPIOA_MODER &= ~(0x3 << (5*2)); // Clear bits
*GPIOA_MODER |= (0x1 << (5*2)); // Set as output

// Set pin 5 high
*GPIOA_ODR |= (1 << 5);
```

Q2.4: Explain the volatile keyword and when to use it

Answer:
volatile tells the compiler that a variable can change unexpectedly (hardware, ISR, or other thread). It prevents compiler optimization that might cache the value.

When to use volatile:
1. Hardware registers
2. Variables modified by ISRs
3. Multi-threaded shared variables
4. Memory-mapped peripherals

Example:

```
volatile uint8_t flag = 0; // Modified by ISR

void ISR_Handler(void) {
flag = 1; // Set by interrupt
}

void main(void) {
while (flag == 0) {
// Wait for interrupt
// Without volatile, compiler might optimize this to infinite loop
}
}
```

Q2.5: What happens when you dereference a NULL pointer?

Answer:
Dereferencing a NULL pointer causes a **segmentation fault** in protected memory systems (Linux) or **hard fault exception** in embedded systems (microcontrollers).

System-level behavior:
• **ARM Cortex-M**: Triggers HardFault_Handler (if memory address 0x00000000 is not mapped)
• **MMU systems**: Operating system terminates the process
• **Bare-metal without MPU**: May read/write address 0x0 if valid memory exists there

Prevention:

```
int *ptr = NULL;

// Always check before dereferencing
if (ptr != NULL) {
*ptr = 10;
}

// Set to NULL after freeing
free(ptr);
ptr = NULL; // Prevents double-free and use-after-free
```

3. BITWISE OPERATIONS

Essential for Embedded Systems

Bitwise operations are fundamental in embedded programming for configuring hardware registers, manipulating individual bits, and implementing efficient algorithms. You must master these for interviews.

Q3.1: Explain all bitwise operators with examples

Answer:

Operator	Name	Example	Result
&	AND	0b1100 & 0b1010	0b1000
|	OR	0b1100 | 0b1010	0b1110
^	XOR	0b1100 ^ 0b1010	0b0110
~	NOT	~0b1100	0b0011
<<	Left Shift	0b0011 << 2	0b1100
>>	Right Shift	0b1100 >> 2	0b0011

Q3.2: How do you set, clear, and toggle a specific bit?

Answer:
These are the most common bit operations in embedded systems:

```
// Set bit n (make it 1)
value |= (1 << n);

// Clear bit n (make it 0)
value &= ~(1 << n);

// Toggle bit n (flip it)
value ^= (1 << n);

// Check if bit n is set
if (value & (1 << n)) {
// Bit is set
}

// Example: Configure GPIO Pin 5
#define PIN 5
volatile uint32_t *GPIO_ODR = (uint32_t*)0x40020014;

*GPIO_ODR |= (1 << PIN); // Set pin HIGH
*GPIO_ODR &= ~(1 << PIN); // Set pin LOW
*GPIO_ODR ^= (1 << PIN); // Toggle pin
```

Q3.3: Write a function to count the number of set bits (Hamming Weight)

Answer:

```
// Method 1: Simple loop
int count_set_bits(uint32_t n) {
int count = 0;
while (n) {
count += n & 1;
n >>= 1;
}
return count;
}

// Method 2: Brian Kernighan's Algorithm (faster)
```

```
int count_set_bits_fast(uint32_t n) {
int count = 0;
while (n) {
n &= (n - 1); // Clears rightmost set bit
count++;
}
return count;
}

// Example: count_set_bits(0b1011) returns 3
```

Q3.4: How do you swap two numbers without using a temporary variable?

Answer:

```
// Using XOR (works for all types, no overflow)
void swap_xor(int *a, int *b) {
if (a != b) { // Avoid swapping same variable
*a = *a ^ *b;
*b = *a ^ *b;
*a = *a ^ *b;
}
}

// Using arithmetic (may overflow)
void swap_add(int *a, int *b) {
*a = *a + *b;
*b = *a - *b;
*a = *a - *b;
}

// Example:
int x = 5, y = 10;
swap_xor(&x, &y);
// x = 10, y = 5
```

Q3.5: Write macros for bit manipulation

Answer:

```
// Bit manipulation macros
#define SET_BIT(REG, BIT) ((REG) |= (1U << (BIT)))
#define CLEAR_BIT(REG, BIT) ((REG) &= ~(1U << (BIT)))
#define TOGGLE_BIT(REG, BIT) ((REG) ^= (1U << (BIT)))
#define READ_BIT(REG, BIT) (((REG) >> (BIT)) & 1U)
#define WRITE_BIT(REG, BIT, VAL) \
((VAL) ? SET_BIT(REG, BIT) : CLEAR_BIT(REG, BIT))

// Multi-bit operations
#define SET_BITS(REG, MASK) ((REG) |= (MASK))
#define CLEAR_BITS(REG, MASK) ((REG) &= ~(MASK))
#define MODIFY_BITS(REG, MASK, VAL) \
((REG) = ((REG) & ~(MASK)) | ((VAL) & (MASK)))

// Usage:
SET_BIT(GPIOA->ODR, 5); // Set bit 5
MODIFY_BITS(TIM1->CR1, 0xFF, 0x05); // Modify lower byte
```

4. DATA STRUCTURES FOR EMBEDDED SYSTEMS

Resource-Constrained Design

Embedded systems require efficient, predictable data structures. Unlike desktop applications, you must consider memory footprint, deterministic behavior, and stack/heap usage.

Q4.1: Implement a circular buffer (ring buffer)

Answer:
Circular buffers are essential for UART, I2C, and other communication protocols in embedded systems.

```
typedef struct {
uint8_t buffer[256];
uint8_t head; // Write index
uint8_t tail; // Read index
} CircularBuffer;

void cb_init(CircularBuffer *cb) {
cb->head = 0;
cb->tail = 0;
}

int cb_write(CircularBuffer *cb, uint8_t data) {
uint8_t next_head = (cb->head + 1) % 256;
if (next_head == cb->tail) {
return -1; // Buffer full
}
cb->buffer[cb->head] = data;
cb->head = next_head;
return 0;
}

int cb_read(CircularBuffer *cb, uint8_t *data) {
if (cb->head == cb->tail) {
return -1; // Buffer empty
}
*data = cb->buffer[cb->tail];
cb->tail = (cb->tail + 1) % 256;
return 0;
}

int cb_is_empty(CircularBuffer *cb) {
return (cb->head == cb->tail);
}

int cb_is_full(CircularBuffer *cb) {
return ((cb->head + 1) % 256 == cb->tail);
}
```

Q4.2: Why avoid malloc() in embedded systems? What are alternatives?

Answer:
Problems with malloc():
• Non-deterministic execution time
• Heap fragmentation over time
• Memory leaks in long-running systems
• Requires larger memory overhead
• Not suitable for safety-critical systems

Alternatives:

```
// 1. Static allocation
static uint8_t buffer[1024];

// 2. Memory pools
#define POOL_SIZE 10
typedef struct {
```

```
uint8_t data[64];
uint8_t in_use;
} MemBlock;

static MemBlock memory_pool[POOL_SIZE];

MemBlock* pool_alloc() {
for (int i = 0; i < POOL_SIZE; i++) {
if (!memory_pool[i].in_use) {
memory_pool[i].in_use = 1;
return &memory_pool[i];
}
}
return NULL; // Pool exhausted
}

void pool_free(MemBlock *block) {
block->in_use = 0;
}

// 3. Stack-based allocation
void function() {
uint8_t local_buffer[256]; // On stack
// Use buffer...
} // Automatically freed
```

Q4.3: Implement a linked list without malloc()

Answer:

```
typedef struct Node {
int data;
struct Node *next;
} Node;

#define MAX_NODES 50
static Node node_pool[MAX_NODES];
static uint8_t node_used[MAX_NODES];

Node* alloc_node() {
for (int i = 0; i < MAX_NODES; i++) {
if (!node_used[i]) {
node_used[i] = 1;
return &node_pool[i];
}
}
return NULL;
}

void free_node(Node *node) {
int index = node - node_pool;
if (index >= 0 && index < MAX_NODES) {
node_used[index] = 0;
}
}

// Usage
Node *head = NULL;
Node *new_node = alloc_node();
if (new_node) {
new_node->data = 42;
new_node->next = head;
head = new_node;
}
```

5. MICROCONTROLLER BASICS

Q5.1: Explain the difference between microprocessor and microcontroller

Answer:

Feature	Microprocessor	Microcontroller
CPU	Only CPU	CPU + Peripherals
Memory	External RAM/ROM	On-chip RAM/ROM/Flash
I/O	Requires external chips	Built-in GPIO, ADC, Timers
Power	Higher	Lower (sleep modes)
Cost	Higher	Lower
Example	Intel i7, ARM A72	STM32, AVR, PIC

Q5.2: What are GPIO modes and how do you configure them?

Answer:
GPIO Modes:
• Input: Read digital signals
• Output: Drive digital signals
• Alternate Function: UART, SPI, I2C
• Analog: ADC input

```
// STM32 GPIO Configuration Example
#define RCC_AHB1ENR (*(volatile uint32_t*)0x40023830)
#define GPIOA_MODER (*(volatile uint32_t*)0x40020000)
#define GPIOA_ODR (*(volatile uint32_t*)0x40020014)

// Enable GPIOA clock
RCC_AHB1ENR |= (1 << 0);

// Configure PA5 as output (LED)
GPIOA_MODER &= ~(0x3 << (5*2)); // Clear
GPIOA_MODER |= (0x1 << (5*2)); // Set as output (01)

// Toggle LED
GPIOA_ODR ^= (1 << 5);
```

Q5.3: Explain interrupt handling in microcontrollers

Answer:
Interrupts allow MCU to respond to events immediately without polling. When an interrupt occurs:
1. Current execution stops
2. Processor state saved to stack
3. Interrupt Service Routine (ISR) executes
4. Return from ISR, restore state

Example - External Interrupt:

```
volatile uint8_t button_pressed = 0;

// ISR - Keep it short!
void EXTI0_IRQHandler(void) {
if (EXTI->PR & (1 << 0)) { // Check pending bit
button_pressed = 1;
EXTI->PR |= (1 << 0); // Clear pending bit
}
}

int main(void) {
```

```
    // Configure interrupt...
    NVIC_EnableIRQ(EXTI0_IRQn);

    while (1) {
    if (button_pressed) {
    button_pressed = 0;
    // Handle event in main loop
    }
    }
    }
```

ISR Best Practices:

- Keep ISRs short and fast
- Set flags, don't do processing
- Never use blocking calls (delay, printf)
- Be careful with shared variables (use volatile)
- Clear interrupt flags to prevent re-triggering

6. RTOS FUNDAMENTALS

Q6.1: What is an RTOS and when should you use it?

Answer:
A Real-Time Operating System (RTOS) manages multiple tasks with deterministic timing guarantees. It provides task scheduling, synchronization, and inter-task communication.

Use RTOS when:
• Multiple independent tasks with different priorities
• Need for deterministic response times
• Complex synchronization requirements
• Managing multiple I/O devices simultaneously

Don't use RTOS when:
• Simple, single-purpose application
• Limited RAM (< 8KB)
• Timing requirements are relaxed
• Bare-metal is sufficient

Q6.2: Explain task scheduling algorithms

Answer:

Algorithm	Description	Use Case
Preemptive	Higher priority task preempts lower	Hard real-time
Round-Robin	Equal time slices for same priority	Fairness needed
Cooperative	Tasks yield voluntarily	Simple systems
Priority-based	Highest priority runs first	Most RTOS

Q6.3: What is priority inversion and how do you prevent it?

Answer:
Priority Inversion: High-priority task waits for resource held by low-priority task, while medium-priority task runs.

Scenario:
1. Low-priority task acquires mutex
2. High-priority task needs same mutex, blocked
3. Medium-priority task preempts low-priority task
4. High-priority task starves!

Solutions:
Priority Inheritance: Temporarily boost low-priority task to high-priority
Priority Ceiling: Raise task priority to highest of all tasks using that resource

FreeRTOS Example:

```
    // Create mutex with priority inheritance
    SemaphoreHandle_t xMutex = xSemaphoreCreateMutex();

    // High-priority task
    void vHighPriorityTask(void *pvParameters) {
    xSemaphoreTake(xMutex, portMAX_DELAY);
    // Critical section
    xSemaphoreGive(xMutex);
    }
```

7. COMMUNICATION PROTOCOLS

Q7.1: Compare UART, SPI, and I2C

Answer:

Feature	UART	SPI	I2C
Wires	2 (TX, RX)	4 (MOSI, MISO, SCK, CS)	2 (SDA, SCL)
Speed	115200 bps typical	10+ Mbps	100-400 kHz
Topology	Point-to-point	Master-Slave(s)	Multi-master
Complexity	Simple	Medium	Complex
Distance	Long	Short	Short
Use Case	Debug, GPS	SD cards, displays	Sensors, EEPROMs

Q7.2: Implement UART transmission

Answer:

```
// UART registers (example)
#define USART2_SR (*(volatile uint32_t*)0x40004400)
#define USART2_DR (*(volatile uint32_t*)0x40004404)
#define TXE_FLAG (1 << 7) // Transmit data register empty

void uart_send_char(char c) {
// Wait until TX buffer is empty
while (!(USART2_SR & TXE_FLAG));
USART2_DR = c; // Send character
}

void uart_send_string(const char *str) {
while (*str) {
uart_send_char(*str++);
}
}

// Usage
uart_send_string("Hello World\r\n");
```

Q7.3: Explain SPI communication with timing diagram

Answer:
SPI is a synchronous full-duplex protocol with 4 signals:
• **MOSI**: Master Out Slave In
• **MISO**: Master In Slave Out
• **SCK**: Serial Clock
• **CS/SS**: Chip Select (active low)

SPI Modes:
Mode 0: CPOL=0, CPHA=0 (Clock idle low, sample on rising edge)
Mode 1: CPOL=0, CPHA=1 (Clock idle low, sample on falling edge)
Mode 2: CPOL=1, CPHA=0 (Clock idle high, sample on falling edge)
Mode 3: CPOL=1, CPHA=1 (Clock idle high, sample on rising edge)

Basic SPI Transfer:

```
uint8_t spi_transfer(uint8_t data) {
SPI2->DR = data; // Write data
while (!(SPI2->SR & SPI_SR_RXNE)); // Wait for receive
return SPI2->DR; // Read received data
}
```

```
// Read from device
CS_LOW(); // Select device
spi_transfer(READ_CMD);
uint8_t data = spi_transfer(0x00); // Dummy byte to read
CS_HIGH(); // Deselect
```

8. ADVANCED TOPICS

Q8.1: Explain watchdog timer and its importance

Answer:
A watchdog timer (WDT) resets the system if software fails to refresh it within a timeout period. This prevents system hang in case of software crashes or infinite loops.

```
// Independent Watchdog (IWDG) - STM32
#define IWDG_KR (*(volatile uint32_t*)0x40003000)
#define IWDG_PR (*(volatile uint32_t*)0x40003004)
#define IWDG_RLR (*(volatile uint32_t*)0x40003008)

void watchdog_init(void) {
IWDG_KR = 0xCCCC; // Start watchdog
IWDG_KR = 0x5555; // Enable write access
IWDG_PR = 0x06; // Prescaler /256
IWDG_RLR = 4095; // Max reload value (~32s timeout)
IWDG_KR = 0xAAAA; // Refresh
}

void watchdog_refresh(void) {
IWDG_KR = 0xAAAA; // Pet the dog
}

int main(void) {
watchdog_init();
while (1) {
// Main loop
watchdog_refresh(); // Must call regularly
}
}
```

Q8.2: What are DMA and its advantages?

Answer:
Direct Memory Access (DMA) transfers data between memory and peripherals without CPU intervention.

Advantages:
• CPU freed for other tasks
• Faster data transfers
• Lower power consumption
• Supports burst transfers

Example - ADC DMA:

```
uint16_t adc_buffer[100];

void adc_dma_init(void) {
// Configure DMA
DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR; // Peripheral
DMA1_Channel1->CMAR = (uint32_t)adc_buffer; // Memory
DMA1_Channel1->CNDTR = 100; // Number of transfers
DMA1_Channel1->CCR |= DMA_CCR_MINC // Memory increment
| DMA_CCR_CIRC // Circular mode
| DMA_CCR_EN; // Enable
}

// CPU can do other work while DMA fills buffer
```

Q8.3: Explain memory barriers and cache coherency

Answer:
Memory barriers ensure proper ordering of memory operations, critical when dealing with:
• Multi-core systems
• DMA operations

- Memory-mapped peripherals
- Cache coherency

```
// ARM Cortex-M barrier instructions
__DMB(); // Data Memory Barrier
__DSB(); // Data Synchronization Barrier
__ISB(); // Instruction Synchronization Barrier

// Example: Ensure write completes before DMA
*peripheral_reg = value;
__DSB(); // Wait for write to complete
DMA_Start(); // Now safe to start DMA
```

9. PRACTICAL PROBLEM SOLVING

Q9.1: Design a debouncing circuit in software

Answer:

```
// Method 1: Simple delay-based debouncing
#define DEBOUNCE_TIME_MS 50

uint8_t button_read(void) {
static uint8_t last_state = 0;
static uint32_t last_change = 0;
uint8_t current_state = GPIO_READ(BUTTON_PIN);

if (current_state != last_state) {
last_change = get_tick_ms();
last_state = current_state;
}

if ((get_tick_ms() - last_change) > DEBOUNCE_TIME_MS) {
return current_state;
}
return last_state;
}

// Method 2: State machine debouncing
typedef enum {
IDLE,
PRESSED,
RELEASED
} ButtonState;

ButtonState debounce_button(void) {
static uint8_t counter = 0;
static ButtonState state = IDLE;

if (GPIO_READ(BUTTON_PIN) == 0) { // Pressed
if (counter < 10) counter++;
if (counter >= 10 && state != PRESSED) {
state = PRESSED;
return PRESSED;
}
} else { // Released
if (counter > 0) counter--;
if (counter == 0 && state != IDLE) {
state = IDLE;
return RELEASED;
}
}
return state;
}
```

Q9.2: Implement a software PWM

Answer:

```
#define PWM_PERIOD 100 // 100 steps
static uint8_t duty_cycle = 50; // 50% duty
static uint8_t counter = 0;

// Call this from timer ISR (e.g., 1kHz)
void software_pwm_isr(void) {
counter++;
if (counter >= PWM_PERIOD) {
counter = 0;
}

if (counter < duty_cycle) {
GPIO_SET(PWM_PIN); // High
} else {
GPIO_CLEAR(PWM_PIN); // Low
}
}
```

```
void set_pwm_duty(uint8_t duty) {
if (duty <= 100) {
duty_cycle = duty;
}
}

// Usage
set_pwm_duty(75); // 75% duty cycle
```

10. INTERVIEW STRATEGY & TIPS

How to Approach Technical Questions

1. **Clarify the Question**: Ask about constraints, expected inputs/outputs
2. **Think Aloud**: Explain your thought process
3. **Start Simple**: Basic solution first, then optimize
4. **Consider Edge Cases**: NULL pointers, buffer overflow, integer overflow
5. **Explain Trade-offs**: Memory vs speed, simplicity vs features

Common Pitfalls to Avoid

• Don't memorize solutions - understand concepts
• Never say "I don't know" - always try to reason through
• Don't write code without planning first
• Avoid overconfidence - acknowledge what you don't know
• Don't ignore the interviewer's hints

System-Level Thinking

Interviewers want to see that you understand:
• How compiler translates code to assembly
• What happens at hardware level
• Memory layout (stack, heap, .text, .data, .bss)
• Startup sequence (boot, clock config, main)
• Power consumption trade-offs

Key Topics to Review Before Interview

Topic	Key Points
Pointers	All types, hardware register access, volatile
Bit Operations	Set/clear/toggle, masks, shifts
Memory	Stack vs heap, static, alignment, endianness
Interrupts	ISR rules, priority, nested, atomicity
RTOS	Tasks, semaphores, mutex, queues
Communication	UART, SPI, I2C timing and configuration
Debugging	Tools, techniques, common bugs

Practice Problems

1. Write a function to reverse bits in a byte
2. Implement producer-consumer using circular buffer
3. Design state machine for UART protocol
4. Calculate timer values for specific baud rate
5. Debug a race condition in interrupt code
6. Optimize power consumption for battery device
7. Implement CRC calculation

8. Design bootloader architecture

Final Words

Remember: Embedded systems interviews test both your knowledge and your ability to apply it. Don't just memorize - understand the 'why' behind every concept. Build small projects to gain hands-on experience. The best engineers are those who can explain complex topics simply and think at the system level.

Good luck with your interviews!